웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10 쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번 [202210282] 이름: [고세윤] 제출일: [10/28]

서비스 이름 [운동할래?]

기술 조합과 버전 [React ,FastAPI /react-dom@19.3.0,react@19.3.0,vite@8.3.4, Python 3.12.0, FastAPI 0.143.0]

GitHub 저장소 URL [https://github.com/seyoon7211/JWT-Assignment.git]

소스 코드 기준 [브랜치 / 코드 구현 기준 커밋 ID 를 기록할 수 있습니다. 최종 제출 기준 커밋 SHA 는 LMS 에 제출합니다.]

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

[이 서비스는 축구, 야구, 테니스처럼 함께 운동할 사람이 필요한 사용자를 위한 인원 모집 커뮤니티입니다. 사용자는 모집 글에 운동 종목, 일정, 장소, 필요한 인원 등을 작성하고, 다른 사용자는 로그인 없이 모집 내용을 확인할 수 있습니다. 프론트엔드는 로그인 상태에 따라 화면과 버튼을 변경하기 위해 React 를 선택했고, 백엔드는 Python 으로 API 를 구현할 수 있는 FastAPI 를 사용하였다. 제공 템플릿의 게시판 CRUD 와 데이터 저장 구조는 유지하고, JWT 발급·검증, 로그인 연결, 작성자 권한 검사, 로그아웃 및 오류 처리를 구현했습니다. 모집 정보는 별도 입력 항목을 추가하지 않고 게시글 제목과 본문에 작성하도록 구성했습니다.]

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	[완료]	[backend-fastapi/app.py, frontend-react/src/main.jsx]	[작성]
로그인과 JWT 발급	[완료]	[backend-fastapi/app.py, auth.py, users.py, frontend-react/src/api.js]	[작성]
인증과 내 정보 조회	[완료]	[backend-fastapi/auth.py, app.py, frontend-react/src/api.js]	[작성]
본인 글 작성·수정·삭제	[완료]	[backend-fastapi/auth.py, app.py, store.py, frontend-react/src/api.js, main.jsx]	[작성]

기능	완료 여부	관련 파일	비고 또는 제한
만료·로그아웃·오류 표시	[완료]	[backend-fastapi/auth.py, frontend-react/src/api.js, main.jsx]	[작성]
DB 데이터 유지	[완료]	[backend-fastapi/store.py, data/board.sqlite]	[작성]

웹서비스설계 02

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

[React 의 api.js 에서 api 요청을 보내면 FastAPI 서버로 전달된다. 서버는 요청에 필요한 인증을 검사하고 store.py 를 통해 데이터를 처리한뒤 JSON 응답을 보낸다.

로그인 요청 흐름은 이메일과 비밀번호를 입력하면 POST /api/auth/login 요청이 전송된다. 서버는 users.py 의 verify_credentials()로 계정과 비밀번호를 확인하고, 인증에 성공하면 auth.py 의 issue_token()에서 JWT 를 발급한다. 이후 토큰, 유효 시간, 공개 사용자 정보를 반환하며, 프론트엔드는 api.js 의 session 변수에 토큰과 사용자 정보를 메모리로 저장하고 화면에 로그인한 사용자 이름을 표시한다.

게시글수정 요청 흐름은 작성자가 수정 내용을 입력하고 저장하면 api.js 에서 Authorization: Bearer <JWT> 헤더를 첨부하여 PUT /api/posts/id 요청을 전송한다. 서버는 current_user_id()에서 JWT 를 검증하고 사용자를 확인한 뒤 게시글의 존재 여부를 확인한다. 이어서 assert_owner()가 현재 사용자 ID 와 게시글 작성자 ID 를 비교하여 본인 글인지 검사하고, 제목과 내용의 입력값을 검증한다. 모든 검사를 통과하면 store.py 를 통해 SQLite 의 게시글을 수정하고, 수정된 글을 반환하여 화면을 갱신한다.]

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-fastapi/auth.py / issue_token(), current_user_id(), assert_owner()	인증·권한 검사 함수의 기본 틀을 제공한다.	[JWT 발급, 토큰 검증, 작성자 확인 부분을 채운다/본인 작성 및 AI 작성]	[로그인 사용자와 글 작성자를 확인하기 위해 구현한다.]
frontend-react/src/api.js / login(), authorizationHeaders(), request()	API 요청의 기본 구조를 제공한다.	[로그인 응답을 저장하고 요청에 토큰을 첨부한다. 401 응답이면 로그인 상태를 해제한다. 본인 작성 디버그 AI]	[서버 인증과 브라우저 로그인 상태를 연결한다]
frontend-react/src/main.jsx	게시글 목록과 입력 폼 등 기본 화면 제공	게시판의 타이틀과 소제목등을 변경	게시판 목적에 따른

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[win 25H2/ 새 폴더 설치 / .env.example 파일을 복사하여 .env 를 만들고 난수키를 생성하여 JWT_SECRET 에 저장함 / npm ci , npm run build,npm run preview, http://127.0.0.1:4173 에서 빌드된 게시판 화면이 정상적으로 표시됨./ py -m venv .venv,.venv\Scripts\python.exe -m pip install -r requirements.txt,.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000 FastAPI 서버가 8000 번 포트에서 실행됨./ backend-fastapi/data/board.sqlite]

[4173 번 접속 및 API 연결 확인 / 서버 재시작 후 데이터 유지 / docs/evidence/T1, docs/evidence/T15,T15-1]

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수: [backend-fastapi/app.py 의 login(), auth.py 의 issue_token()]

코드 발췌

```
def issue_token(user):
  payload = {
    "sub": str(user["id"]),
    "exp": int(time.time()) + TOKEN_TTL,
    "iss": os.getenv("JWT_ISSUER", "webservice-class"),
    "aud": os.getenv("JWT_AUDIENCE", "board-api"),
  }

  return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

동작과 응답 코드 설명

[로그인 요청으로 이메일과 비밀번호를 받는다. app.py 의 fields()에서 입력을 검사하고, verify_credentials()로 계정과 비밀번호를 확인한다. 계정이 맞으면 issue_token()을 호출한다.토큰에는 문자열 사용자 ID 인 sub, 만료 시각인 exp, 발급자 iss, 사용 대상 aud 를 넣는다. HS256 과 서버 비밀 키로 서명하며 기본 유효 시간은 90 초다. 비밀번호는 토큰에 넣지 않는다.성공하면 **200** 과 함께 accessToken, expiresIn, user 를 반환한다. 입력이 잘못되면 fields()에서 **400 / BAD_INPUT**, 계정이 맞지 않으면 if not user 분기에서 **401 / INVALID_CREDENTIALS** 를 반환한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: [frontend-react/src/api.js 의 login(), authorizationHeaders(), clearSession()

코드 발췌

```
function authorizationHeaders() {
 if (!session.accessToken) {
  return {};
 }
 return {
  Authorization: `Bearer ${session.accessToken}`,
 };
}

export async function login(email, password) {
 const data = await request("/auth/login", {
  method: "POST",
  body: JSON.stringify({ email, password }),
 });
 session = {
  accessToken: data.accessToken,
  user: data.user,
 };
 return session.user;
}
```

동작과 응답 코드 설명

[로그인에 성공하면 응답의 토큰과 사용자 정보를 session 변수에 저장한다. 브라우저 메모리만 사용하며 localStorage 와 sessionStorage 는 사용하지 않는다. 새로고침하면 로그인 정보가 사라진다.토큰이 있으면 Authorization: Bearer <JWT> 헤더를 붙인다. 토큰이 없으면 빈 객체를 반환하여 인증 헤더를 보내지 않는다. 로그아웃하면 clearSession()으로 토큰과 사용자 정보를 지운다. 정상 토큰으로 내 정보를 조회하면 **200** 을 받는다. request()에서 **401** 을 받으면 세션을 지우고 오류를 전달한다. **403** 은 권한 부족이므로 로그인 상태를 지우지 않는다. 오류 안내에는 서버가 반환한 code 와 message 를 사용한다.]

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: [backend-fastapi/auth.py 의 current_user_id()]

코드 발췌

```
payload = jwt.decode(
    parts[1],
    SECRET_KEY,
    algorithms=["HS256"],
    issuer=os.getenv("JWT_ISSUER", "webservice-class"),
    audience=os.getenv("JWT_AUDIENCE", "board-api"),
    options={
        "require": ["sub", "exp", "iss", "aud"],
        "verify_exp": False,
    },
)

if payload["exp"] <= time.time():
    raise ApiError(
        401, "TOKEN_EXPIRED", "로그인이 만료되었습니다."
    )

return user_id
```

동작과 응답 코드 설명

[요청 헤더에서 Bearer 토큰을 꺼내고 jwt.decode()로 서명을 검증한다. 허용 알고리즘은 HS256 으로 제한하며 sub, exp, iss, aud 가 있는지 확인한다. 발급자와 사용 대상도 설정값과 비교한다. 이후 사용자 ID 와 만료 시각의 형식을 검사하고, find_public()으로 실제 계정이 있는지 확인한다. verify_exp=False 로 둔 것은 만료 검사를 생략하려는 것이 아니라 아래 조건문에서 직접 처리하기 위해서다. 검증을 통과한 사용자 ID 만 다음 처리에 반환한다. Bearer 헤더가 없거나 형식이 틀리면 **401 / TOKEN_REQUIRED** 를 반환한다. 서명이나 클레임이 잘못되면 예외 처리에서 **401 / INVALID_TOKEN**, 정상 토큰이 만료됐으면 만료 분기에서 **401 / TOKEN_EXPIRED** 를 반환한다. 검증 성공 후 /api/me 는 **200** 과 공개 사용자 정보를 반환한다.

.

3 4 작성자 권한 검사

파일 / 함수: [backend-fastapi/auth.py 의 assert_owner(), app.py 의 create(), update(), delete()]

코드 발췌

```
def assert_owner(user_id, author_id):
  if user_id != author_id:
    raise ApiError(
      403,
      "FORBIDDEN",
      "본인이 작성한 글만 수정하거나 삭제할 수 있습니다.",
    )
```

동작과 응답 코드 설명

현재 로그인한 사용자 ID 와 DB 에서 읽은 글의 authorId 를 비교한다. 두 값이 다르면 403 / FORBIDDEN 을 반환하고 수정이나 삭제를 실행하지 않는다. 같으면 다음 처리를 진행한다. 글을 등록할 때 작성자는 요청 본문이 아니라 current_user_id()가 반환한 ID 로 정한다. 등록·수정 입력에 authorId 나 userId 를 추가하면 fields()에서 400 / BAD_INPUT 으로 거부한다. 401 은 토큰 인증 실패이고, 403 은 인증은 됐지만 다른 사람의 글이라 권한이 없는 경우다. 정상적으로 본인 글을 수정하면 200, 삭제하면 본문 없이 204 를 반환한다. 프론트에서도 본인 글에만 버튼을 표시하지만 실제 차단은 서버에서 수행한다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: [WIN/ node.js v22.20.0/ 크롬 / .venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000,npm run preview]

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: [10/28]

토큰 TTL: [90 / 3]

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	[로그아웃 상태에서 게시글 확인]	[200,로그인 없이 게시글 목록 조회 가능]	[200/해당없음]	[PASS]	[docs/evidence/T01.png]
T02	[Alice 계정으로 로그인]	[200,로그인 성공]	[200/해당없음]	[PASS]	[docs/evidence/T02.png]
T03	[잘못된 로그인]	[401 INVALID_CREDENTIALS, 로그인 실패]	[401/ INVALID_CREDENTIALS]	[PASS]	[docs/evidence/T03.png,T03-1.png]
T04	[토큰없이 /ME 와 글쓰기]	[401 TOKEN_REQUIRED, 두 요청 모두 실패]	[401/ TOKEN_REQUIRED]	[PASS]	[docs/evidence/T04.png]
T05	[Alice 토큰으로 글 작성 후 /me 조회]	[글 작성 201, 내 정보 조회 200·Alice 정보]	[201/해당없음 200/해당없음]	[PASS]	[docs/evidence/T05.png]
T06	[Alice 토큰으로 본인 글 수정·삭제]	[200,204/글 정상 삭제]	[200/해당없음,204/해당없음]	[PASS]	[docs/evidence/T06.png, docs/evidence/T06-1.png]
T07	[Alice 글을 Bob 의 정상 토큰으로 수정]	[403 FORBIDDEN, 수정 불가]	[403/FORBIDDEN]	[PASS]	[docs/evidence/T07.png]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T08	[Alice 글을 Bob 의 정상 토큰으로 삭제]	[403 FORBIDDEN, 삭제불가]	[403/FORBIDDEN]	[PASS]	[docs/evidence/T08.png]
T09	[유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청]	[401 INVALID_TOKEN, 데이터 변경 없음]	[401/INVALID_TOKEN]	[PASS]	[docs/evidence/T09.png]
T10	[실제 만료 뒤 /me 및 변경 요청]	[401 TOKEN_EXPIRED, 화면 로그인 해제와 안내]	[401/ TOKEN_EXPIRED]	[PASS]	[docs/evidence/T10.png]
T11	[로그인 후 로그아웃 후 /me]	[401 TOKEN_REQUIRED,토큰이 없어 정보 조회 불가]	[401 TOKEN_REQUIRED]	[PASS]	[docs/evidence/T11.png]
T12	[Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가]	[400 BAD_INPUT, 위조 글 미생성]	[400/BAD REQUEST]	[PASS]	[docs/evidence/T12.png]
T13	[본인 토큰으로 빈 제목·공백 본문·길이 초과 입력을 각각 전송]	[400 BAD_INPUT, DB 변경 없음]	[400/BAD REQUEST]	[PASS]	[docs/evidence/T13.png]
T14	[없는 양의 정수 ID 의 조회·수정·삭제 및 잘못된 ID 조회]	[없는 글 404, 잘못된 ID 400]	[404 NOT FOUND/400 BAD REQUEST]	[PASS]	[docs/evidence/T14.png]
T15	[서버 종료.재시작후에도 글이 남아있는지 확인]	[남아있음]	[남아있음]	[PASS]	[docs/evidence/T15.png,T15-1.png]
T16	[로그인 후 페이지 새로고침]	[로그아웃 되고 게시글 보임]	[로그아웃 되고 게시글 보임]	[PASS]	[docs/evidence/T16.png,T16-1.png]
T17	[토큰 없이 수정,삭제/bob 으로 수정,삭제]	[토큰이 없을 땐 실패 401,bob 일때는 작업 성공]	[토큰 없을 때 :401/TOKEN REQUIRED,BOB: 204/해당없음]	[판정]	[docs/evidence/T17.png]

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	[200]	[dos/evidence/T01.png]
새 글 등록	**201 및 Location**	[201]	[dos/evidence/T05.png]
본인 글 삭제	**204 및 빈 본문**	[204]	[dos/evidence/T17-1.png]]
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	[400/BAD_REQUEST]	[dos/evidence/T12.png]]
잘못된 계정	**401 INVALID_CREDENTIALS**	[401/INVALID_CREDENTIALS]	[dos/evidence/T03-1.png]
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	[401/TOKEN_REQUIRED]	[dos/evidence/T04.png]
변조·클레임 오류	**401 INVALID_TOKEN**	[401/INVALID_TOKEN]	[dos/evidence/T09.png]]
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	[401/ TOKEN_EXPIRED]	[dos/evidence/T18.png]]
다른 작성자의 변경	**403 FORBIDDEN**	[404_NOT_FOUND]	[dos/evidence/T07.png]]
없는 글	**404 NOT_FOUND**	[404_NOT_FOUND]	[dos/evidence/T06-1.png]]

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

[증거 1:doc/evidence/T02.png.]

다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

[:doc/evidence/T07.png]

토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[:doc/evidence/T10.png]

자동 검사와 로컬 배포 증거

[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로

docs/evidence/bulid.png,/docs/evidence/api test.png, docs/evidence/T01.png/

.\backend-fastapi\.venv\Scripts\python.exe .\tools\check_api.py --mode final 를 사용하여 로그인 ,공개조회등 40 개의 항목을 검사]

웹서비스설계 05

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: [사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
[chatgpt]	[학습 ,개발 환경 셋팅, 과제 진행 질의,코드 첨삭 및 코드 생성, 보고서 작성 보조 등]	[테스트 powershell 명령어 생성 하여 테스트를 진행, current_user_id(request) 함수 생성, 동작원리 첨삭 등]	[**그대로 반영/수정 후 반영**]

대표 사례

도구 / 날짜: [chatgpt/10.24]

요청한 내용: [BE-02: Bearer 추출 → 서명/알고리즘/만료/클레임 검증 → 실제 사용자 확인. + 과제 안내의 BE02 부분을 넣고 구현을 부탁함]

AI 가 제안한 내용: [코드를 제공함]

실제 반영 위치: [def current_user_id(request)]

내가 확인하고 수정한 내용: [X]

검증 방법과 결과: [주어진 코드를 ai 에게 설명을 부탁하여 BE02 가 요구하는 요구 사항이 잘 들어있는지 확인함]

도구 / 날짜: [chatgpt/10.28]

요청한 내용: [테스트 체크리스트 복사 후 테스트 진행을 도와 달라고함]

AI 가 제안한 내용: [차례대로 테스트 하는 방법과 react 가 아닌 터미널에서 진행할 때 명령어를 제공]

실제 반영 위치: [보고서 테스트 부분]

내가 확인하고 수정한 내용: [수정 X 테스트 코드 그대로 터미널에 작성 후 결과 관찰]

검증 방법과 결과: [주어진 명령어를 터미널에 입력 후 나온 결과와 원래 생각하고 있던 결과와의 비교 후 테스 결과 확정]

도구 / 날짜: [chatgpt/10.28]

요청한 내용: [3-2 의 동작원리가 제대로 쓴지 확인과 추가하거나 불필요한 부분 삭제 요청]

AI 가 제안한 내용: [잘못된 부분을 삭제하고 부족한 부분을 써줌]

실제 반영 위치: [보고서 3-2 동작원리]

내가 확인하고 수정한 내용: [ai 가 제안한대로 잘못 쓴 부분을 고침]

검증 방법과 결과: [ai 에게 내가 잘못 이해하고 있는 부분을 설명 받음]

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
[]	[파일/함수]	[작성]

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

[사용자가 aurhhorld 를 다른 사람의 id 로 보내 작성자를 위조할수있다. 요청 본문의 작성자가 아닌 jwt 를 검증한 사용자 id 를 작성자로 저장하여 막는다.]

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

[버튼 없이도 powershell 로 직접 수정이나 삭제를 요청할 수 있기 때문이다.assert_owner()에서 검사한다.]

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

[decode 는 토큰 내용을 읽고,werify 는 믿을 수 있는 토큰인지 검사한다.사용자 ID 형식과 실제 계정 존재 여부를 확인한다.]

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

[로그아웃하면 현재 탭의 토큰과 사용자 정보를 지운다.토큰이 만료되면 로그인 상태가 해제된다.새로고침하면 로그인 정보는 사라지지만 게시글은 유지된다.복사돈 토큰은 새로고침이나 로그아웃에 사라지지 않고 만료전까지 사용 가능하다.]

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

[만료 토큰 테스트를 준비하던 중 게시글 조회에서 404 NOT_FOUND 가 발생했다. 원인을 확인하니 PowerShell 의 $post 변수에 이전 글 ID 가 남아 있었고, 해당 글은 이미 삭제된 상태다. 시험 글을 새로 작성하고 새 ID 로 변수와 비교 데이터를 다시 설정하여 해결했다. 이후 공개 조회가 정상 동작하는지 확인하고 만료 토큰 검사를 이어서 진행했다. 토큰 문제로 생각했지만 실제로는 테스트에 사용한 데이터와 변수의 문제였다는 점에서 원인을 찾기 어려웠다.]

현재 남은 문제와 다음에 개선할 점: [없으면 없음으로 표시]

제출 확인: [O] 실제 코드와 설명 일치 [0] 테스트 직접 수행 [0] AI 활용 및 출처 기록 [0] 비밀 키와 토큰 제거

GitHub 확인: [0] 루트 README 작성 [0] 보고서 PDF 와 증거 업로드 [0] README 문서 링크 확인 [0] 교수자 접근 가능 [0] 저장소 URL 과 커밋 SHA 제출